JOB MARKET ANALYSIS
USING DATA SCIENCE AND MACHINE LEARNING

PROJECT REPORT



Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Technology in Computer Science and Engineering

Submitted by: Sachin Raj
Registration Number: 12419763

Department of Computer Science and Engineering
Lovely Professional University

Under the guidance of : **Dr. Mrinalini Rana**
Academic Session: 2026

STUDENT DECLARATION

I declare that the project report entitled “Job Market Analysis Using Data Science and Machine Learning” has been prepared as part of my academic work. The report describes the dataset, exploratory data analysis, visual analysis, and machine-learning experiments performed for understanding job listings and classifying job categories from job titles.

The analysis and interpretation included in the final submitted version should reflect the actual dataset and program outputs. Any third-party dataset, software library, documentation, or other material used in the project must be acknowledged in the references.

I understand that the final report must not contain fabricated measurements, results, or screenshots. Where a result or screenshot has not yet been inserted, a clearly marked space has been provided for completion.

Student signature: _____

Date: _____

CERTIFICATE

This is to certify that the project report entitled “Job Market Analysis Using Data Science and Machine Learning” is submitted by Sachin Raj, student of Bachelor of Technology in Computer Science and Engineering at Lovely Professional University, for academic evaluation.

The report documents the project objectives, data preparation, exploratory analysis, visualizations, and supervised classification experiments. The final version should be reviewed against the executed Python program and the original dataset before approval.

Project Guide: _____

Designation: _____

Signature: _____

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the faculty members of the Department of Computer Science and Engineering at Lovely Professional University for their guidance and academic support. Their teaching has helped me understand the principles of data analysis, programming, and machine learning.

I also acknowledge the maintainers of the dataset used in this project and the developers of the open-source Python ecosystem. Libraries such as pandas, NumPy, Matplotlib, and scikit-learn provide essential tools for data preparation, visualization, and model development.

Finally, I thank my family, friends, and classmates for their encouragement throughout the completion of this project.

ABSTRACT

Job portals contain large collections of job listings that can be studied to understand occupational demand, hiring activity, workplace arrangements, and the skills employers request. This project applies data science and machine-learning methods to a job-listings dataset consisting of four CSV files: jobs.csv, jobs_by_company.csv, open_jobs_daily.csv, and skill_demand_daily.csv.

The work is organised around exploratory data analysis and seven objectives: examining the distribution of job categories, identifying categories with fresher-friendly listings, reviewing companies with fresher roles, studying daily open-job trends, identifying demanded skills, comparing remote and non-remote listings, and predicting a job category from its title. The Python workflow uses pandas for tabular data processing, Matplotlib for visualisation, and scikit-learn for text vectorisation and classification.

The supervised-learning task treats job title as the input feature and category as the target label. CountVectorizer converts text into numerical features. Logistic Regression, Multinomial Naive Bayes, Decision Tree, Random Forest, and K-Nearest Neighbours (KNN) are evaluated using a held-out test set. Accuracy and numerical confusion matrices are used for comparison. The final report should include the measured output generated from the exact dataset and code used.

The report provides dedicated spaces for the supplied visualisations, output tables, the decision-tree diagram, the accuracy comparison chart, and the complete final Python program. Findings should be interpreted in the context of the dataset's coverage, collection period, definitions, and possible class imbalance.

TABLE OF CONTENTS

Update this table in Microsoft Word after the report is finalised: References → Table of Contents → Automatic Table.

1. Introduction
2. Source of Dataset
3. Exploratory Data Analysis (EDA) Process
4. Analysis on Dataset
  4.1 Job Category Distribution
  4.2 Fresher-Friendly Jobs by Category
  4.3 Companies Offering Fresher Roles
  4.4 Daily Open Job Trend
  4.5 Most Demanded Skills
  4.6 Remote vs Other Job Listings
  4.7 Job Category Prediction from Job Title
5. Conclusion
6. Future Scope
7. References
Appendix A. Complete Python Code
Appendix B. Additional Screenshots and Execution Evidence

LIST OF FIGURES

Figure 1. Top 10 Job Categories
Figure 2. Top Categories for Fresher-Friendly Jobs
Figure 3. Top 10 Companies Offering Fresher Roles
Figure 4. Daily Open Job Trend
Figure 5. Daily Open Jobs: All vs Fresher-Friendly
Figure 6. Top 10 Most Demanded Skills
Figure 7. Remote vs Other Job Listings
Figure 8. Decision Tree Classifier
Figure 9. Accuracy Comparison of Five ML Models
Figure 10 onward. Additional screenshots, code execution, and output evidence

LIST OF TABLES

Table 1. Dataset files and their analytical purpose
Table 2. Project objectives and outputs
Table 3. Software and libraries used
Table 4. Data-quality checks
Table 5. Classification-model comparison
Table 6. Results collection sheet

ABBREVIATIONS

EDA — Exploratory Data Analysis
CSV — Comma-Separated Values
ML — Machine Learning
NLP — Natural Language Processing
TF/Term Count — Frequency of a token in a text field
X — Input feature matrix
y — Target labels
TP — True Positive
FP — False Positive
FN — False Negative
TN — True Negative
KNN — K-Nearest Neighbours
LR — Logistic Regression
NB — Naive Bayes
DT — Decision Tree
RF — Random Forest

1. INTRODUCTION

1.1 Background

Online recruitment platforms publish job advertisements across many occupations, industries, experience levels, and working arrangements. Each listing may contain a job title, category, employer, remote-work indicator, or other descriptive fields. Analysing these listings can help summarise the composition of advertised opportunities and identify patterns that may be useful to students, job seekers, recruiters, and researchers.

Data science provides a systematic approach to transforming raw listings into useful evidence. The process typically includes importing the data, understanding its schema, checking data quality, cleaning inconsistent values, summarising variables, and communicating findings through visualisations. Machine learning extends the analysis by learning patterns from labelled examples and using those patterns to classify new observations.

1.2 Project Overview

This project studies the supplied job-listings dataset and applies descriptive analysis and text classification. The dataset is represented by four CSV files: jobs.csv, jobs_by_company.csv, open_jobs_daily.csv, and skill_demand_daily.csv. Each file supports a different analytical view: individual job listings, company-level fresher roles, daily open-job counts, and skill-demand summaries.

1.3 Problem Statement

A large collection of job listings can be difficult to interpret by manually reviewing individual records. The project therefore seeks to organise job data into meaningful summaries and evaluate whether a job category can be predicted from the wording of its title. The task is constrained by the fields and observations actually available in the dataset.

1.4 Aim

The aim is to perform exploratory data analysis on job listings and develop a basic supervised machine-learning workflow for predicting job categories from job titles.

1.5 Objectives

- Analyse the distribution of job listings across categories.
- Identify categories containing the greatest number of fresher-friendly jobs, where the dataset provides a suitable indicator.
- Identify companies with the largest reported number of fresher roles.
- Examine the daily trend in open job listings.
- Rank skills by their reported job demand.
- Compare remote and non-remote job listings.
- Train and compare five classifiers that predict category from job title.

1. INTRODUCTION (CONTINUED)

1.6 Scope

The scope covers structured CSV analysis, basic data cleaning, descriptive summaries, static charts, text vectorisation, train-test evaluation, and comparison of five classification algorithms. It does not claim to represent every job in the labour market. Conclusions are limited to the records and definitions in the source data.

1.7 Intended Users

- Students and fresh graduates exploring the types of roles represented in the dataset.
- Recruitment analysts interested in category and skill counts.
- Researchers are learning how to apply EDA and text classification to real-world tabular data.
- Developers seeking a small baseline for job-title categorisation.

1.8 Project Workflow

- Locate and download the dataset and retain its source information.
- Load each CSV file with pandas and inspect dimensions, columns, data types, and sample rows.
- Check missing values, duplicates, inconsistent text, and invalid or unexpected values.
- Prepare separate analysis subsets and calculate counts or time-series summaries.
- Create visualisations for the seven project objectives.
- Prepare job titles as text features and category labels as the target.
- Train the selected classifiers and calculate evaluation metrics.
- Compare outputs, document limitations, and preserve screenshots and final code.

1.9 Expected Deliverables

- A reproducible Python script.
- EDA summaries and visualisations for the stated objectives.
- Classification accuracy and numerical confusion matrices for five models.
- A visual decision-tree representation and model-accuracy comparison chart.
- A report containing methods, results, limitations, screenshots, and references.

2. SOURCE OF DATASET

2.1 Dataset Source

The project uses a dataset described as “Job Listings: Open Jobs in Every Field (Daily)” and obtained from Kaggle. Before submission, enter the exact dataset page URL, author or uploader, download date, dataset version, and any licence or usage terms. These details should be copied from the actual dataset page rather than guessed.

https://www.kaggle.com/datasets/chandranshbinjola/job-listings/data

2.2 Dataset Files

The working dataset is organised into four CSV files. The intended analytical role of each file is summarised below. Confirm the actual column names and definitions by inspecting the files before finalising this section.

File name	Expected role	Project use
jobs.csv	Individual job listings	Category counts, fresher-friendly listings, remote comparison, title classification
jobs_by_company.csv	Company-level summaries	Ranking companies by reported fresher-role counts
open_jobs_daily.csv	Daily time-series summary	Trend in open jobs and fresher-friendly jobs, if available
skill_demand_daily.csv	Skill-level demand summary	Ranking the most frequently reported skills

2.3 Data Format

CSV is a plain-text tabular format in which each record is usually stored on a separate line and fields are separated by commas. pandas can read these files into DataFrame objects. The exact encoding, delimiter, date format, and missing-value representation should be verified during import.

2.4 Data Provenance and Reproducibility

Data provenance records where the dataset came from and how it was transformed. Keep an unchanged copy of the downloaded CSV files, record the download date, and perform cleaning in memory or on clearly named derived files. If the dataset is updated, the analysis may produce different totals or model scores.

2. SOURCE OF DATASET (CONTINUED)

2.6 Dataset Suitability

The dataset is suitable for exploratory analysis because it contains listing or aggregated information that can be counted and compared. It is suitable for text classification only if individual job records include both a usable title field and a category label. If the title or category is missing, empty, or inconsistent, those records require a documented handling decision.

2.7 Limitations of the Source

- The records may represent a platform-specific or time-limited sample rather than the complete labour market.

- Repeated listings or cross-posted advertisements may affect counts if duplicates are present.
- Aggregated files may have been generated from a different date range or filtering process than the individual listings.
- Terms such as “fresher-friendly” and “remote” depend on how the source dataset defines or records them.
- Category labels may be imbalanced or noisy, affecting model performance.

2.8 Source Verification Checklist

- Record the exact dataset page URL and dataset author.
- Record the download date and any version or update details.
- Check the licence and permitted use.
- Confirm all four CSV files are present and can be opened.
- Record row counts, column names, date ranges, and key definitions.
- Keep a copy of the original files separate from cleaned data.

3. EXPLORATORY DATA ANALYSIS (EDA) PROCESS

3.1 Meaning of EDA

Exploratory Data Analysis is the process of inspecting and summarising a dataset before or alongside modelling. It helps reveal the structure of the data, the frequency of categorical values, the range of numeric measurements, possible data-quality problems, and patterns that merit further investigation. EDA combines numerical summaries with visual representations.

3.2 Importing the Required Libraries

The project uses pandas to read and manipulate CSV data, NumPy for numerical operations where needed, Matplotlib for charts, and scikit-learn for machine-learning utilities. The actual import statements should match the final Python script.

Typical imports include pandas, matplotlib.pyplot, CountVectorizer, train_test_split, and the five classifier classes. Library versions may be recorded with Python's version information and package metadata when reproducibility is required.

3.3 Loading the CSV Files

Each CSV file is loaded into a separate DataFrame. File paths must point to the actual location of the files on the user's computer. A relative path is usually easier to reproduce when the Python script and data folder are organised consistently.

The first inspection should include head(), shape, columns, info(), and isnull().sum(). These commands help establish the number of rows and columns, sample values, data types, and missing-value counts.

3.4 Initial Inspection

- head() displays the first few records.
- shape returns the number of rows and columns.
- columns lists the column names.
- info() summarises data types and non-null counts.
- isnull().sum() counts missing values in each column.
- duplicated().sum() counts exact duplicate rows.

3. EDA PROCESS (CONTINUED)

3.5 Data Cleaning

Data cleaning improves consistency without changing the meaning of valid records. The project may remove exact duplicates, trim leading and trailing spaces, standardise text casing for selected fields, and exclude records with unusable title or category values from the classification task. Each transformation should be applied only when supported by the actual data.

3.6 Missing Values

Missing values can affect both counts and models. For a category-prediction task, records without a usable title or target category generally cannot provide a supervised training example. Missing values in unrelated fields should not automatically cause a row to be deleted if the row remains valid for the objective being analysed.

3.7 Duplicate Records

Exact duplicate rows may inflate counts and can cause the same advertisement to be represented more than once. The duplicate policy should be stated clearly. Exact-row duplicate removal is not the same as identifying duplicate advertisements with different IDs or slightly different wording.

3.8 Text Standardisation

Job titles are textual data. Converting titles to strings, trimming spaces, and removing unusable empty titles can reduce avoidable inconsistencies. Over-aggressive text cleaning should be avoided because job titles may contain abbreviations, technologies, levels, or domain-specific terms that help classification.

3.9 Excluding the “other” Category

The current project workflow excludes category values labelled “other” before the relevant category visualisations and model training. This decision narrows the analysis to more specific labels, but it also changes the population being described. The report should state that counts and model results refer to the filtered dataset where this exclusion is applied.

3.10 Train-Test Split

The supervised data is divided into training and test subsets. The training set is used to fit the text representation and classifier, while the test set estimates performance on held-out examples. A fixed random_state supports repeatability. Stratification can help preserve class proportions when every retained class has sufficient examples.

3. EDA PROCESS (CONTINUED)

3.11 Text Vectorisation with CountVectorizer

Most scikit-learn classifiers require numerical input. CountVectorizer converts a collection of text documents into a sparse matrix whose columns represent vocabulary tokens and whose values represent token counts. In this project, each job title is treated as a short document. The vocabulary is learned from the training text and then applied to the test text.

The representation does not fully understand meaning or context. Similar titles can be represented similarly when they share words, while synonyms or unusual abbreviations may be treated as different features.

3.12 Classification Algorithms

- Logistic Regression: a linear classifier that estimates class probabilities from the vectorised text features.
- Multinomial Naive Bayes: a probabilistic classifier commonly used with non-negative token-count features.
- Decision Tree: a sequence of feature-based splits that leads to a predicted class.
- Random Forest: an ensemble of decision trees whose predictions are combined.
- K-Nearest Neighbours: predicts a class using labels of nearby training examples in feature space.

3.13 Evaluation Measures

Accuracy is the proportion of test examples predicted correctly. A confusion matrix records the number of predictions for each actual-versus-predicted class pair. Accuracy is easy to interpret, but it may hide weak performance on minority classes when class frequencies are highly unequal.

Accuracy = (Number of correct predictions) / (Total number of predictions)

For a multiclass confusion matrix, the diagonal contains correct predictions for each class. Off-diagonal values indicate which classes are confused with one another.

3.14 Reproducibility and Record Keeping

Record the dataset version, random seed, package versions, filtering rules, vectoriser settings, model parameters, and test split. Save the final code and output screenshots together. If the script is rerun after changing the data or configuration, update the reported results accordingly.

4.1 ANALYSIS 1: JOB CATEGORY DISTRIBUTION

i. Introduction

This analysis measures how job listings are distributed across the categories represented in jobs.csv. Category counts provide a high-level view of which occupational groups appear most frequently in the available records.

ii. General Description

The input is the individual job-listings DataFrame. A categorical field such as category is grouped and counted. The “other” category is excluded in accordance with the project's stated filtering rule. The chart displays the ten categories with the largest counts, subject to the available data.

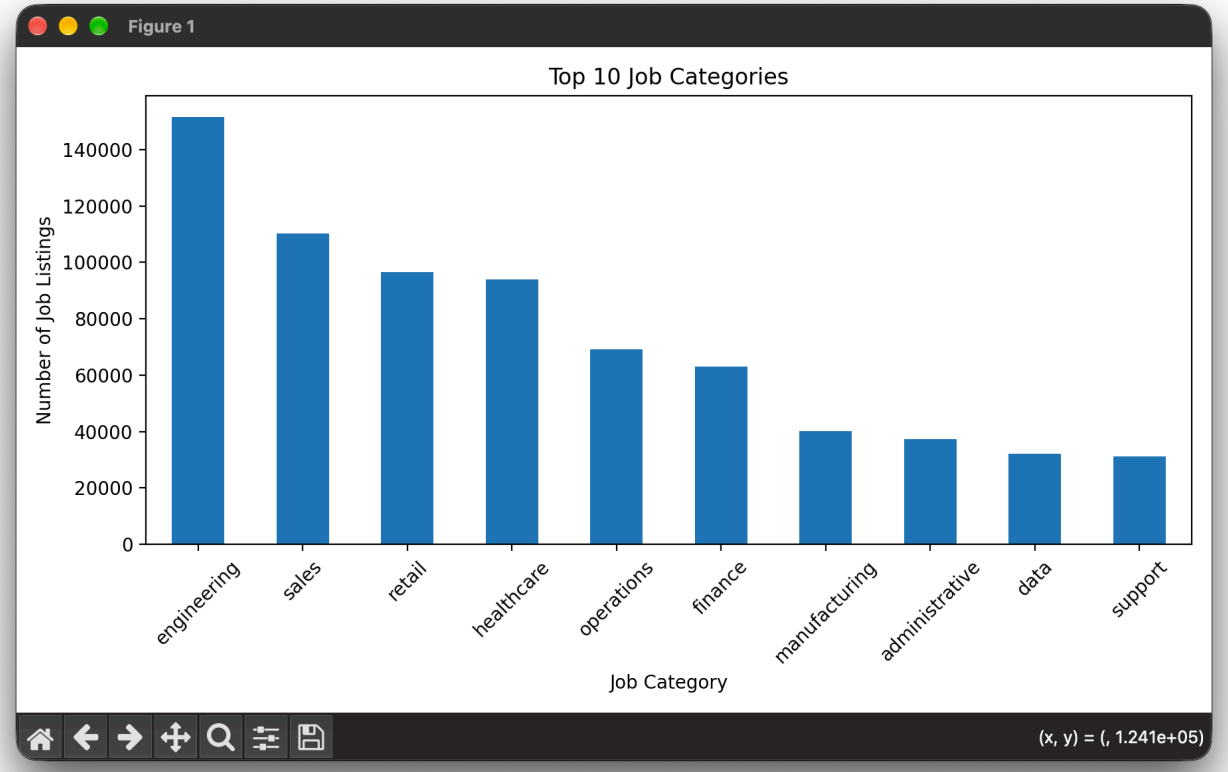
iii. Specific Requirements, Functions and Formulas

Required fields: category. Required operations: remove or filter unusable category values, exclude “other”, count records by category, sort counts in descending order, and select the top ten. A typical pandas operation is value_counts().head(10). The chart uses a bar plot with category on the horizontal axis and listing count on the vertical axis.

iv. Analysis Results

Use the actual generated chart to identify the leading categories and their approximate counts. In the supplied screenshot, engineering is the tallest bar, followed by sales and retail; healthcare and operations also appear among the leading categories. These observations describe the displayed dataset snapshot and should be checked against the final output.

v. Visualization



4.2 ANALYSIS 2: FRESHER-FRIENDLY JOBS BY CATEGORY

i. Introduction

This analysis explores how fresher-friendly opportunities are distributed across job categories. It is intended to help identify categories with relatively more entry-level listings in the dataset.

ii. General Description

The analysis requires a valid indicator for fresher-friendly listings or a documented rule for identifying them. The project uses the relevant field available in jobs.csv and groups qualifying listings by category. The top categories are then plotted as a bar chart.

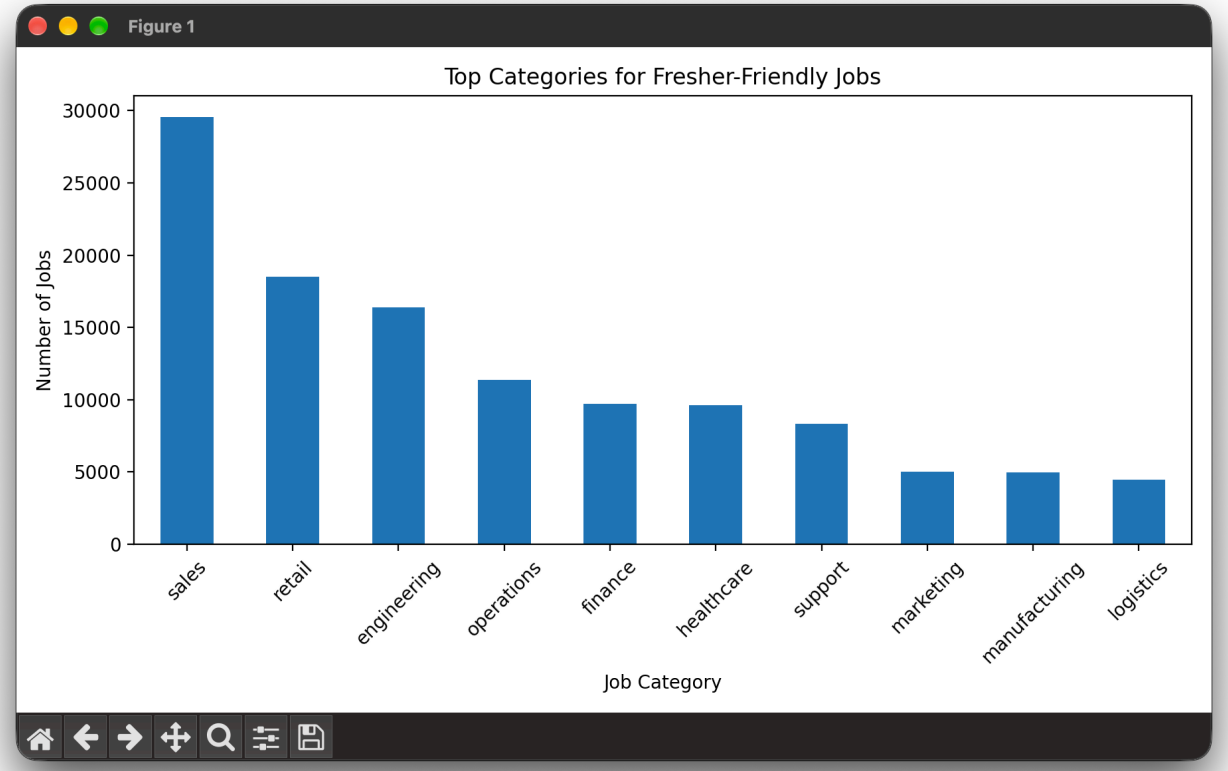
iii. Specific Requirements, Functions and Formulas

Required fields: category and a valid fresher-friendly indicator or equivalent source-defined flag. Filter to records marked as fresher-friendly, exclude “other” where the project rule applies, count records by category, sort descending, and plot the top ten. Do not infer fresher status from job title unless that rule is explicitly implemented and documented.

iv. Analysis Results

The supplied screenshot shows sales as the largest displayed category for fresher-friendly jobs, followed by retail and engineering. Operations, finance, healthcare, and support are also represented. Use the final program output to confirm exact counts and whether the chart uses listing counts or an aggregated measure.

v. Visualization



4.3 ANALYSIS 3: COMPANIES OFFERING FRESHER ROLES

i. Introduction

This analysis ranks companies by the reported number of fresher roles. It provides a company-level view of which employers have the largest counts in the dataset.

ii. General Description

The expected input is jobs_by_company.csv, which should contain a company identifier or name and a numeric count of fresher roles. The data is sorted by the role count and the ten highest values are displayed.

iii. Specific Requirements, Functions and Formulas

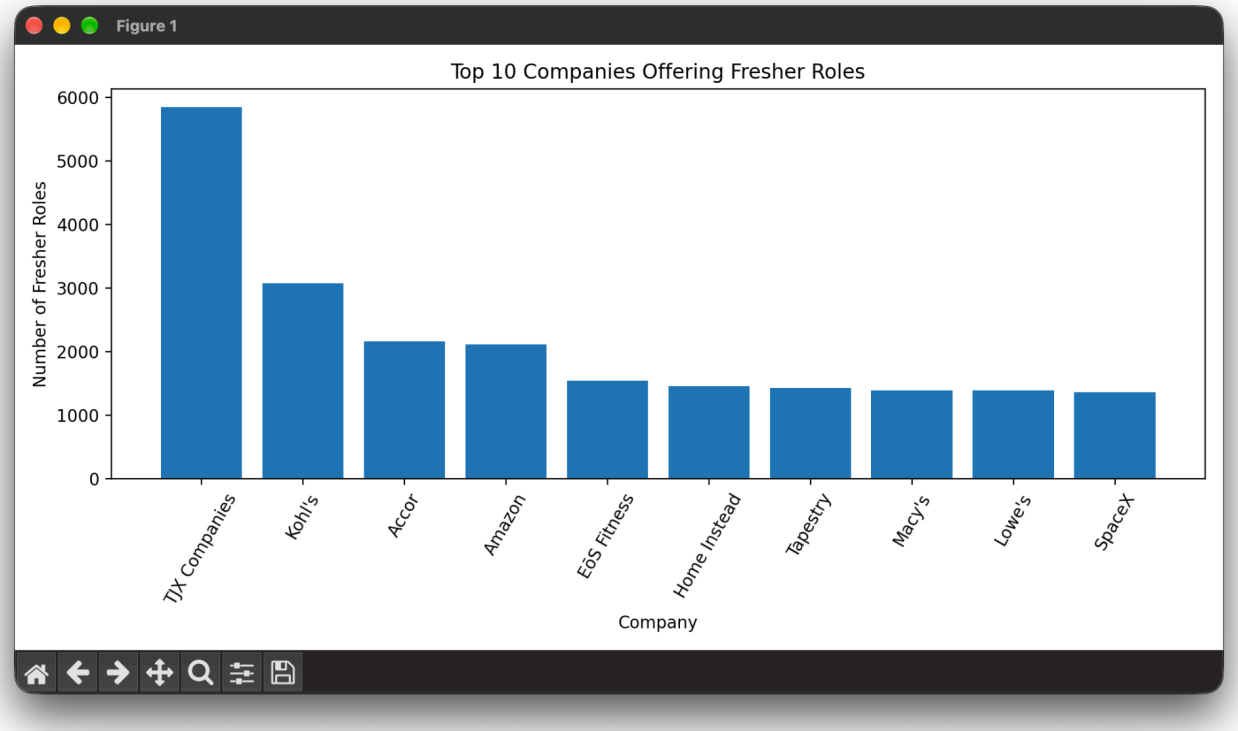
Required fields: company name and fresher-role count. Check that the count field is numeric, handle missing company names according to a documented rule, sort in descending order, select the top ten, and draw a bar chart. If the source contains multiple rows per company, aggregate them only if the data definition supports summation.

iv. Analysis Results

The supplied screenshot shows TJX Companies as the leading company, with Kohl's second, followed by Accor and Amazon. EoS Fitness, Home Instead, Tapestry, Macy's, Lowe's, and SpaceX also appear in the displayed top ten. Confirm exact counts against the final data file.

v. Visualization

Insert the supplied screenshot titled "Top 10 Companies Offering Fresher Roles".



4.4 ANALYSIS 4: DAILY OPEN JOB TREND

i. Introduction

This analysis examines how the total number of open job listings changes across the dates in the daily summary. A time-series chart is appropriate because date order is meaningful.

ii. General Description

The input is open_jobs_daily.csv. It is expected to contain a date field and a measure of open jobs. Dates should be parsed into a datetime type and sorted chronologically before plotting. The chart displays the daily open-job count.

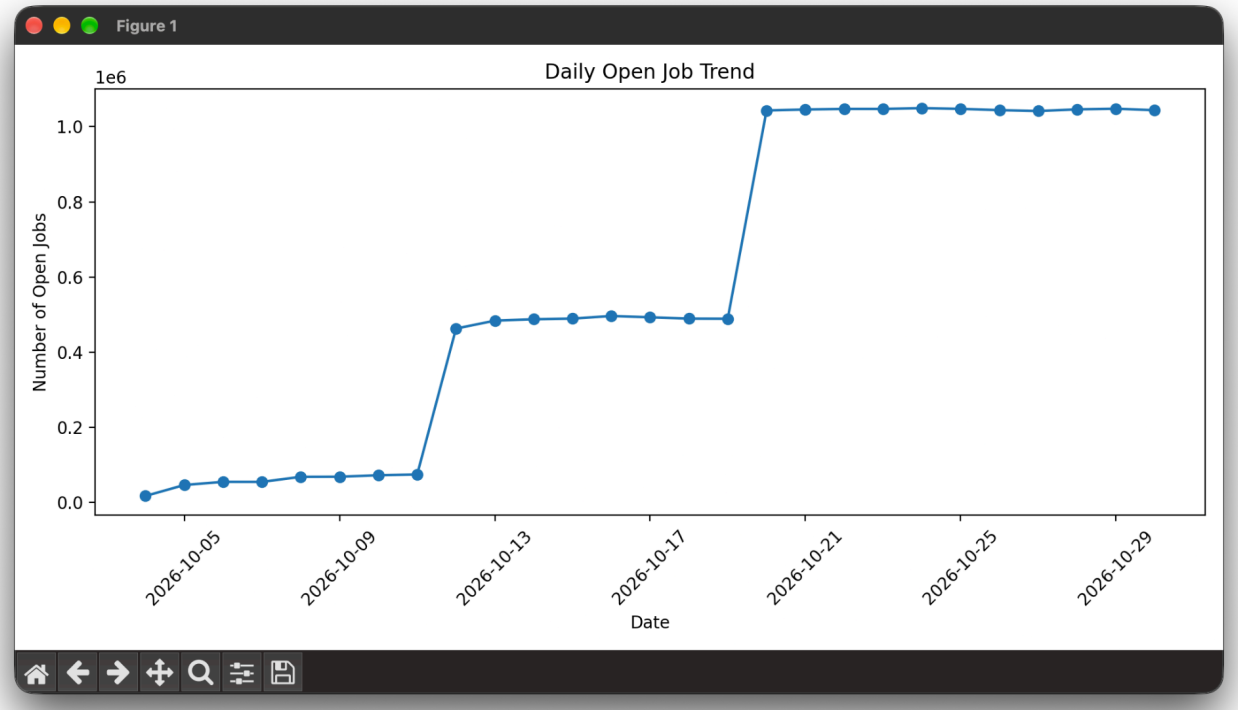
iii. Specific Requirements, Functions and Formulas

Required fields: date and daily open-job count. Convert dates with pandas.to_datetime(), validate numeric counts, sort by date, and plot date against open jobs. If multiple records exist per date, aggregation should follow the source's meaning rather than being applied automatically.

iv. Analysis Results

The supplied chart shows a gradual increase in early October 2026, a sharp step up around mid-October, and another large step around 20 October, followed by a relatively stable level slightly above one million. Such abrupt changes may reflect real changes, ingestion or aggregation updates, or a change in dataset coverage; the chart alone cannot establish the cause.

v. Visualization



4.5 ANALYSIS 5: DAILY OPEN JOBS — ALL VS FRESHER-FRIENDLY

i. Introduction

This analysis compares the daily total of open jobs with the daily subset identified as fresher-friendly. The comparison provides a view of the size and movement of the entry-level subset relative to the total.

ii. General Description

The analysis uses the daily summary and requires both an overall open-job count and a fresher-friendly count aligned to the same dates. A two-series line chart displays the measures over time.

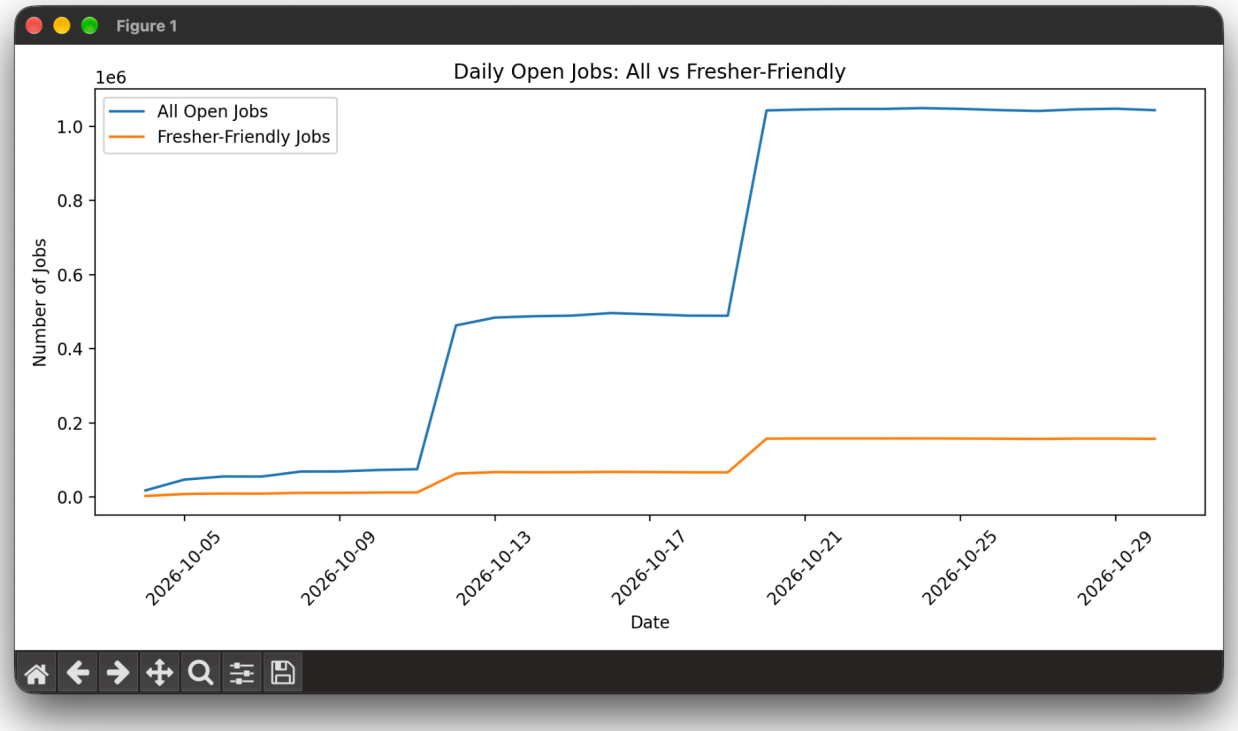
iii. Specific Requirements, Functions and Formulas

Required fields: date, total open jobs, and fresher-friendly open jobs. Parse and sort dates, verify both measures are numeric, and use the same date axis. Do not compare series from different date ranges without stating the limitation.

iv. Analysis Results

In the supplied screenshot, both series show step changes around the same periods as the total daily trend. The fresher-friendly series remains substantially below the total series. The shape suggests that changes in the overall series are mirrored by the fresher-friendly series, but the cause and comparability of the counts require source documentation.

v. Visualization



4.6 ANALYSIS 6: MOST DEMANDED SKILLS

i. Introduction

This analysis ranks skills by their reported job demand. The results can be used to describe which skill labels occur most often in the supplied skill-demand summary.

ii. General Description

The input is skill_demand_daily.csv, expected to contain a skill label and a numeric demand measure. The chart displays the ten highest-ranked skill labels using horizontal bars so that long labels remain readable.

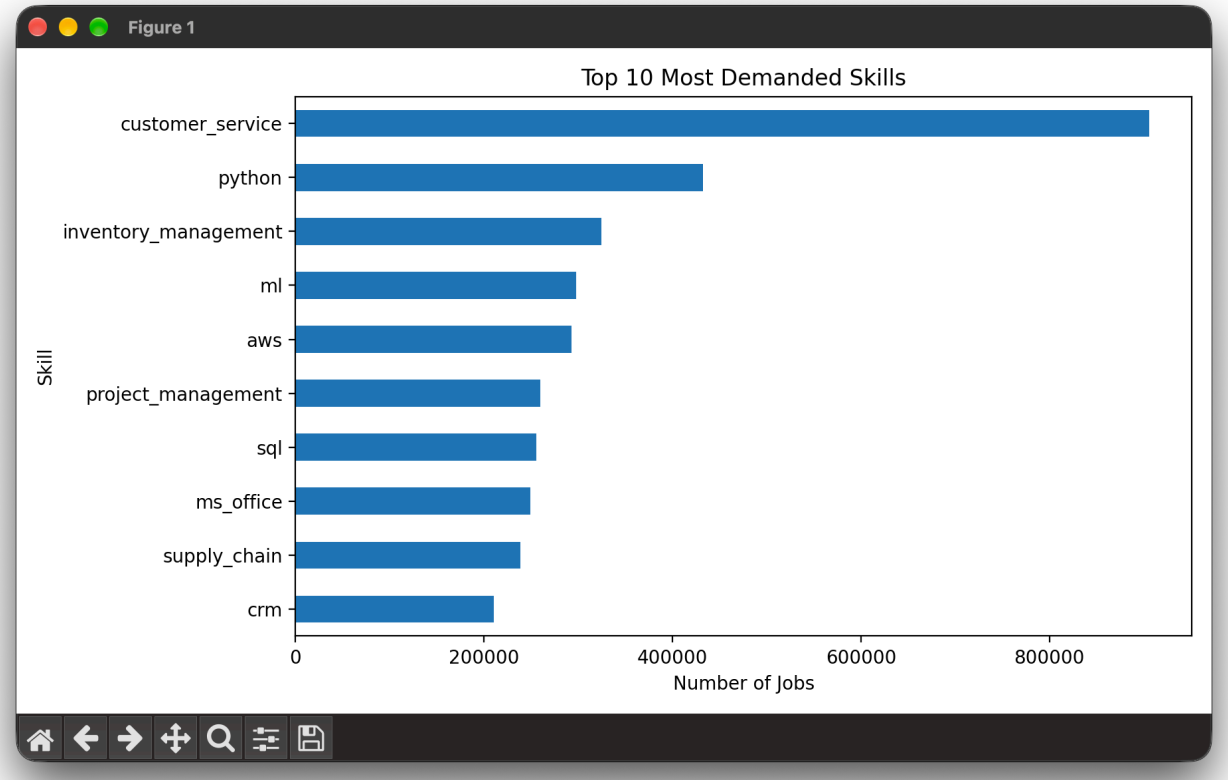
iii. Specific Requirements, Functions and Formulas

Required fields: skill and demand count. Ensure the count is numeric, handle blank skill labels, sort descending, select the top ten, and plot skill against job count. If the file contains multiple dates per skill, the project must state whether the chart uses the latest date, a sum over dates, or another defined aggregation.

iv. Analysis Results

In the supplied screenshot, customer_service has the largest displayed count, followed by python and inventory_management. ML, AWS, project_management, SQL, MS Office, supply_chain, and CRM also appear among the leading labels. The source field's aggregation period must be confirmed before interpreting the values as unique jobs.

v. Visualization



4.7 ANALYSIS 7: REMOTE VS OTHER JOB LISTINGS

i. Introduction

This analysis compares the number of listings marked remote with those not marked remote. It provides a simple summary of the work-location flag in the available listing data.

ii. General Description

The input is jobs.csv and the expected field is a boolean-like remote indicator. The data is grouped by its remote value and counted. The chart compares the False and True groups.

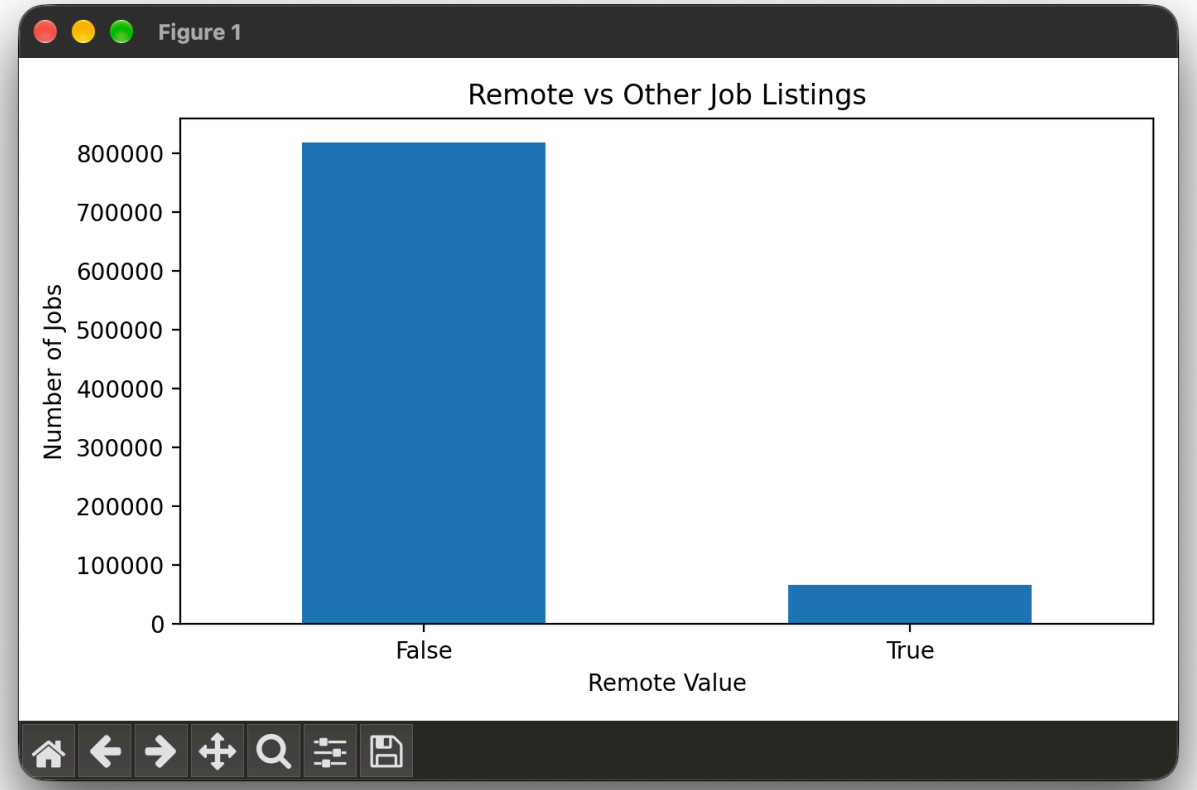
iii. Specific Requirements, Functions and Formulas

Required field: remote. Standardise or inspect values before grouping so that text values such as “true”, “false”, missing values, and boolean values are not silently treated as different groups. Count records by remote status and plot the result. Clarify whether False means explicitly non-remote or simply not flagged as remote.

iv. Analysis Results

The supplied screenshot shows many more listings with remote=False than remote=True. The displayed non-remote/not-flagged group is approximately 820,000 listings, while the remote=True group is roughly 65,000. These are visual approximations from the screenshot; use exact values printed by the final script for the report.

v. Visualization



4.8 ANALYSIS 8: JOB CATEGORY PREDICTION FROM JOB TITLE

i. Introduction

This analysis evaluates whether the category of a job listing can be predicted from its title. It is a supervised multiclass text-classification task: the job title is the input text and the category field is the target label.

ii. General Description

The dataset is cleaned to retain rows with usable titles and categories, and the category labelled “other” is excluded according to the project rule. The workflow uses a train-test split, CountVectorizer to convert titles into token-count features, and five classifiers. The experiment should use the same split and text representation for a fair comparison.

iii. Specific Requirements, Functions and Formulas

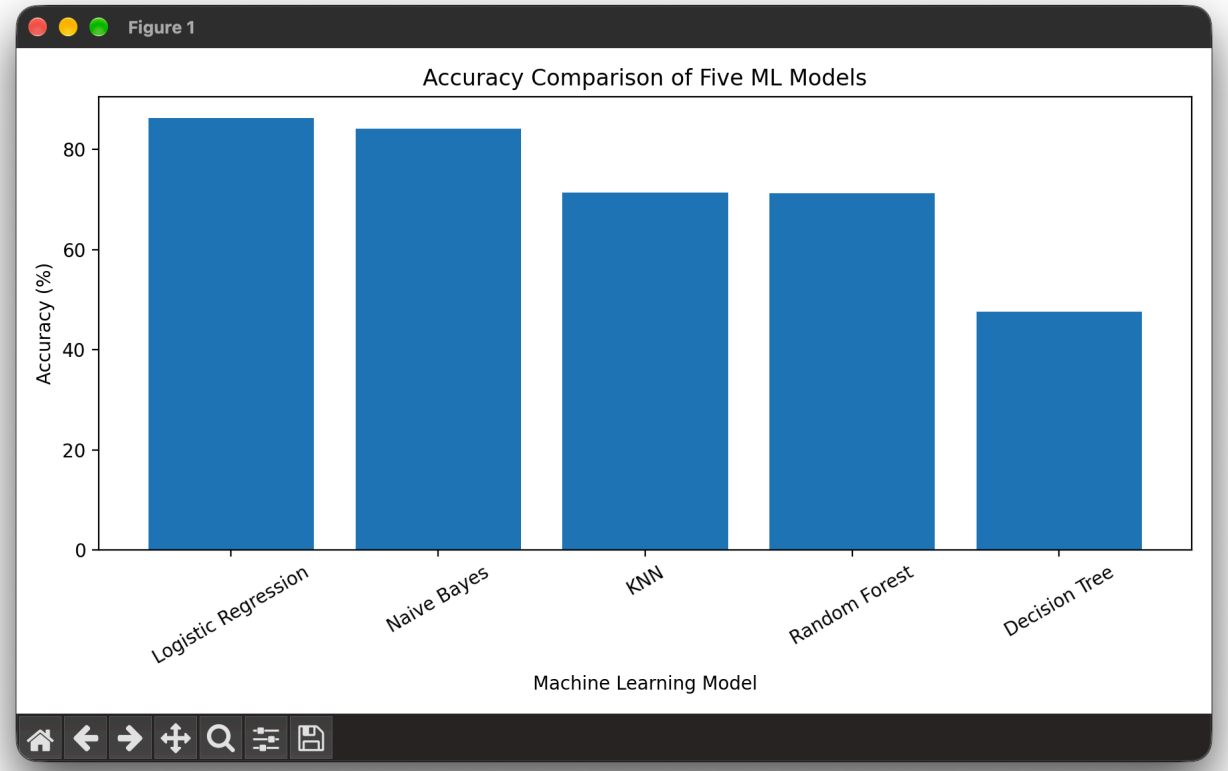
- Input feature: job title.
- Target label: category.
- Text conversion: CountVectorizer.
- Split: train_test_split, with a fixed random_state for reproducibility.
- Models: Logistic Regression, Multinomial Naive Bayes, Decision Tree, Random Forest, and K-Nearest Neighbours.
- Primary metric: accuracy_score(y_test, y_pred).
- Additional diagnostic: confusion_matrix(y_test, y_pred), printed numerically.

Accuracy = Correct predictions ÷ Total test predictions × 100%

iv. Analysis Results

Insert the accuracy reported by the executed program for each model. The supplied accuracy comparison screenshot suggests that Logistic Regression and Naive Bayes perform best among the five displayed models, KNN and Random Forest are lower, and the displayed Decision Tree result is the lowest. The graph indicates approximate values only; use the numerical program output as the authoritative source.

v. Visualization



4.8 ANALYSIS 8 (CONTINUED): CONFUSION MATRICES

A confusion matrix compares actual category labels with predicted labels. In a multiclass problem, each row typically represents an actual class and each column a predicted class, subject to the ordering used by the software. Correct predictions appear on the main diagonal.

The matrices should be copied as numerical output from the final run. Do not replace them with invented values or screenshots from another dataset.

Logistic Regression

```
MODEL 1: LOGISTIC REGRESSION
Accuracy: 0.8624
Accuracy (%): 86.24
Confusion Matrix:
[[ 27   2   1   0   0   0   0   0]
 [  0 140   3   0   0   0  11   4]
 [  0   0  45   1   0   2   5   2]
 [  2   1   0  66   1   2   5   0]
 [  0   1   0   0  37   3   5   1]
 [  0   4   1   0   0  58   1   2]
 [  4   0   1   0   0   3  87   1]
 [  0   4   1   2   1   1   8  79]]
```

Multinomial Naive Bayes

```
MODEL 2: NAIVE BAYES
Accuracy: 0.8416
Accuracy (%): 84.16
Confusion Matrix:
[[ 21   2   0   2   0   0   2   3]
 [  0 152   1   0   0   0   2   3]
 [  0   6  44   1   0   2   0   2]
 [  1   4   0  70   1   1   0   0]
 [  0  11   0   0  30   3   1   2]
 [  1   6   5   0   0  51   1   2]
 [  3   7   0   0   1   2  79   4]
 [  0  11   0   2   0   1   3  79]]
```

Decision Tree

```
MODEL 3: DECISION TREE CLASSIFIER
Confusion Matrix:
[[  0   0   0  29   0   1   0   0]
 [  0 117   0  39   0   0   0   2]
 [  0   0   0  55   0   0   0   0]
 [  0   0   0  71   0   6   0   0]
 [  0   0   0  35   0  12   0   0]
 [  0   0   0  46   0  20   0   0]
 [  0   0   0  77   0   2  17   0]
 [  0   0   0  24   0   0   0  72]]
Accuracy: 0.4752
Accuracy (%): 47.52
```

Random Forest

```
MODEL 4: RANDOM FOREST CLASSIFIER
Accuracy: 0.712
Accuracy (%): 71.2
Confusion Matrix:
[[ 22   6   0   1   0   1   0   0]
 [  0 157   0   0   0   1   0   0]
 [  0  28  19   1   0   2   5   0]
 [  3  12   0  55   0   6   1   0]
 [  0  19   0   0  24   4   0   0]
 [  0  35   0   1   0  30   0   0]
 [  3  22   0   0   0   1  70   0]
 [  0  25   0   2   0   0   1  68]]
```

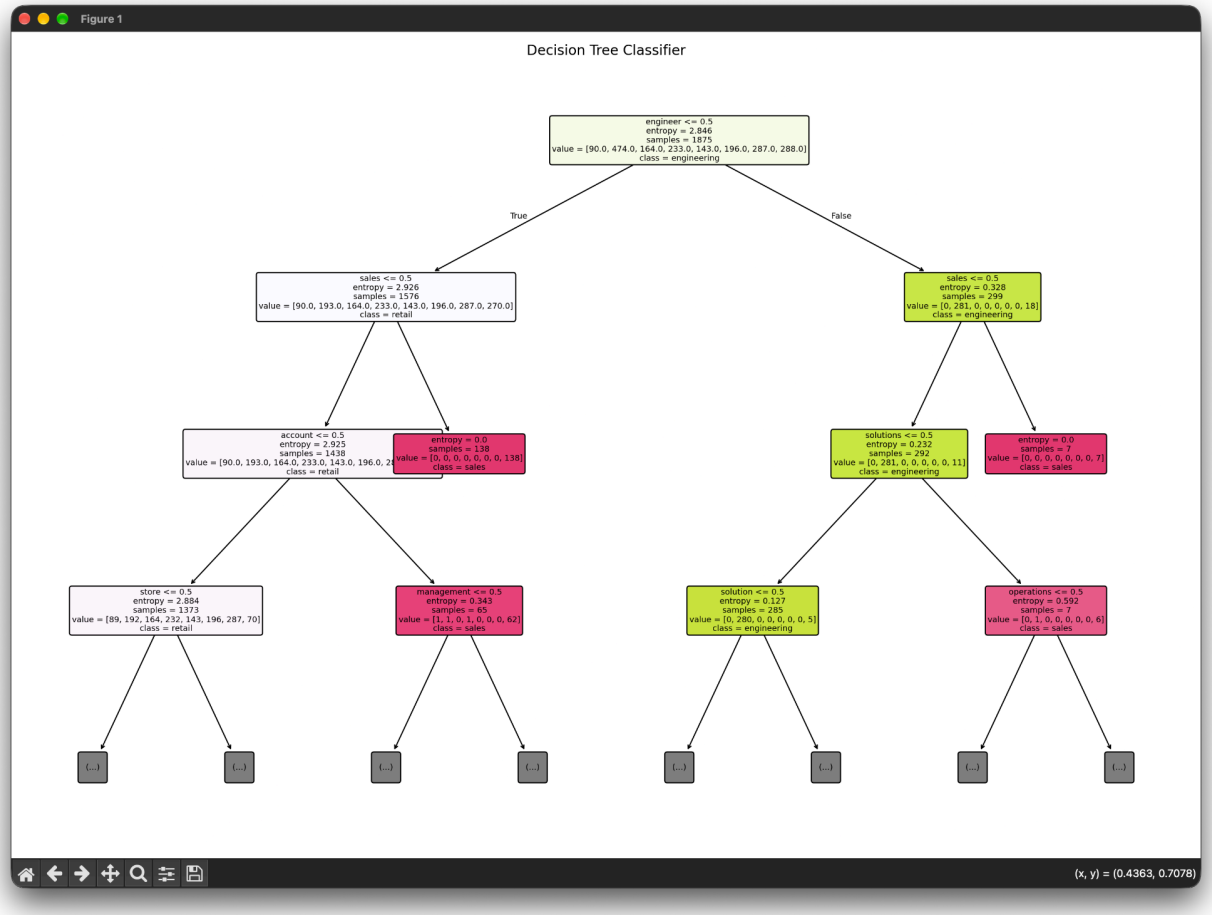
K-Nearest Neighbours

```
MODEL 5: K-NEAREST NEIGHBOURS
Accuracy: 0.7136
Accuracy (%): 71.36
Confusion Matrix:
[[ 26   2   0   1   0   1   0   0]
 [  0 138   2   0   4   0  11   3]
 [  1  11  24   2   7   1   6   3]
 [  5   8   2  46   4   6   6   0]
 [  0   5   2   0  27   5   7   1]
 [  2   5   4   0   5  47   1   2]
 [  3   1   0   3   6   9  72   2]
 [  1  11   1   3   4   2   8  66]]
```

4.8 ANALYSIS 8 (CONTINUED): DECISION TREE

A decision tree classifies examples by applying a sequence of learned feature-based splits. Internal nodes represent decision rules, branches represent outcomes, and leaves represent predicted classes. A tree can be visualised to inspect some of the learned structure, but a large text-feature tree can become difficult to interpret.

The diagram should be generated by the actual Decision Tree model trained in this project. The supplied screenshot shows a tree with job-title-derived features such as words related to engineering, sales, account, store, and operations. Confirm that the screenshot belongs to the final model and the same filtered dataset before submission.



4.9 MODEL COMPARISON AND RESULT COLLECTION

The five models should be compared using the same train-test split and the same text feature matrix. Differences in accuracy can reflect algorithm behaviour, class balance, vocabulary size, sample size, and model parameters. A single accuracy score does not prove that a model is universally better for all categories.

Model	Accuracy (%)	Confusion matrix saved?	
Logistic Regression	86.24	Yes	
Multinomial Naive Bayes	84.16	Yes	
Decision Tree	47.52	Yes	
Random Forest	71.20	Yes	
KNN	71.36	[Yes	

4.10 DATA QUALITY AND FILTERING DECISIONS

Data quality influences every downstream result. Before counting or training, the project should inspect whether the required fields exist and whether their values are usable. A missing or differently named column can indicate that the downloaded version has a different schema from the version expected by the script.

Filtering decisions should be objective-specific. For example, a row with a missing remote indicator may still have a valid title and category, while a row with no category cannot be used as a labelled example for category classification. Keeping separate subsets avoids unnecessary deletion.

Data quality checks

Confirm expected columns before each analysis.

Inspect missing-value counts and duplicate counts.

Check data types for dates and numeric counts.

Review category labels and boolean-like remote values.

Record any excluded records and the reason for exclusion.

Reporting requirement

Report actual row counts before and after cleaning.

State whether “other” was removed globally or only for particular objectives.

Record the number of labelled rows used in modelling.

4.11 INTERPRETING CATEGORICAL COUNTS

Category and company bar charts are useful for ranking groups, but counts depend on the number of source records and the rules used to define a group. A high count means that a category is common in this dataset; it does not automatically mean that the category has the highest growth rate, salary, quality, or probability of hiring an individual applicant.

Top-N charts omit the groups outside the displayed range. The report should make clear that the figure displays the top ten categories or companies rather than the complete distribution.

Recommended interpretation

Name the largest group shown in the chart.

Compare the leading group with the next one using actual counts.

Mention any large gap or close ranking supported by the data.

State that the result applies to the dataset snapshot.

Potential sources of bias

Platform coverage may favour particular employers or occupations.

Repeated advertisements can increase counts.

Category assignment may be inconsistent.

Aggregation methods may differ between files.

4.12 INTERPRETING TIME-SERIES CHARTS

Time-series charts preserve chronological order and make sudden changes visible. The daily open-job charts contain distinct step changes. A sudden change should be investigated before it is interpreted as a labour-market event. Possible explanations include an update to source coverage, a change in aggregation, backfilled data, or a genuine change in reported listings.

Date parsing and sorting are essential. If dates are treated as strings, they may not sort chronologically in every format. If multiple rows exist per date, aggregation must follow the dataset's documented meaning.

Checks before interpretation

Confirm minimum and maximum dates.

Check whether every day is represented.

Inspect duplicate dates and aggregation rules.

Compare the all-jobs and fresher-friendly series on matching dates.

Look for discontinuities and document them without asserting an unsupported cause.

What the chart cannot establish

A trend in listings is not the same as a trend in actual hires.

A count change does not explain why demand changed.

The available date range may be too short for seasonal conclusions.

4.13 INTERPRETING SKILL-DEMAND RESULTS

Skill-demand charts rank labels based on the numeric measure supplied by the dataset. A label such as customer_service or python should be treated as a source-defined skill label, not necessarily as a normalised skill taxonomy. Similar skills may be represented by different spellings or abbreviations.

Before concluding that a skill is the most demanded, confirm whether the count refers to unique job listings, total daily mentions, a latest-day snapshot, or a sum across multiple dates. Summing a daily time series may count the same listing or skill repeatedly.

Data checks

Inspect duplicate skill labels and spelling variants.

Confirm the meaning and time period of the demand measure.

Check for negative, non-numeric, or missing counts.

Document how multiple dates are aggregated, if applicable.

Practical interpretation

Describe the highest-ranked labels shown.

Distinguish technical skills from business or service skills only if a defensible grouping is used.

Avoid generalising the ranking to all employers or the whole economy.

4.14 INTERPRETING REMOTE-WORK COUNTS

The remote comparison is based on a flag in the job-listings table. A True value commonly indicates that the record is marked remote, but the meaning of False depends on the source. It may mean that the role is explicitly non-remote, or only that a remote flag was not set.

Because the displayed count difference is large, the data definition should be checked before drawing a strong conclusion. Missing or inconsistent values should be handled separately rather than silently included in one group.

Recommended checks

Inspect the unique values in the remote field.

Count missing values separately.

Confirm how text and boolean values are represented.

Report exact group counts and, if useful, their shares of all valid records.

Interpretation boundary

The chart counts listings, not successful hires or employee work arrangements.

The dataset may have incomplete remote-work metadata.

The result may differ across source platforms and time periods.

4.15 TEXT CLASSIFICATION: DATA PREPARATION

Text classification maps a textual input to a predefined class. In this project, the input is a job title and the output is a category label. A title may contain role names, seniority indicators, domain terms, tools, or employer-specific language. These words can provide predictive clues but may also be noisy.

Before vectorisation, the project removes unusable title and category values and applies the stated category filtering rule. It is important to split the data before learning the vocabulary so that information from the test set does not influence the training representation.

Feature and target definition

X is the cleaned job-title text.

y is the corresponding category label.

CountVectorizer learns a vocabulary from training titles.

The trained vectoriser transforms test titles into the same feature space.

Sampling considerations

If a subset is used for computational reasons, document the sample size and selection method.

Use stratification when appropriate and feasible.

Check that every retained category has enough examples for the split.

4.16 LOGISTIC REGRESSION: MODEL NOTES

Logistic Regression is a supervised classification method. Despite its name, it can be used for classification rather than only for predicting continuous values. In a multiclass text task, the implementation estimates class scores or probabilities from the token-count features and selects a class according to its decision rule.

With sparse text features, a linear classifier can be a strong baseline because different words can contribute evidence for different categories. Performance depends on vocabulary, regularisation, class distribution, and the train-test split.

Inputs and output

Input: vectorised job title.

Target: category label.

Output: predicted category for each test title.

Evaluation

Record the exact test accuracy.

Inspect the numerical confusion matrix.

Do not infer that a high overall accuracy means all categories are equally well predicted.

4.17 MULTINOMIAL NAIVE BAYES: MODEL NOTES

Multinomial Naive Bayes is a probabilistic classifier often used for document categorisation with non-negative token counts. It applies Bayes' rule with a simplifying conditional-independence assumption between features given the class. Although the assumption is not literally true for natural language, the method can work well for sparse text features.

The classifier is generally efficient to train and provides a useful comparison with a linear discriminative model. Smoothing helps avoid zero probabilities for words not observed in a class during training.

Inputs and output

Input: CountVectorizer token counts.

Target: category.

Output: predicted class labels.

Evaluation

Report accuracy from the held-out test set.

Compare its confusion matrix with Logistic Regression.

Use the same split and vectorised data for a fair comparison.

4.18 DECISION TREE: MODEL NOTES

A Decision Tree builds a hierarchy of feature-based decisions. Each split aims to separate classes according to a criterion such as information gain or impurity reduction. The resulting structure can be drawn and inspected, although trees trained on many text features may be large and sensitive to the training sample.

A tree can capture non-linear decision boundaries, but an unconstrained tree may overfit. The project should report the parameters actually used and avoid implying that every visible split is a general rule that will work on all future listings.

Interpretation

The root node shows the first split selected by the model.

Branches represent outcomes of a split.

Leaf nodes contain final class predictions.

The class distribution in a node indicates which labels are represented.

Evaluation

Record test accuracy and confusion matrix.

Compare the result with the other four classifiers.

Use the exact trained model to generate the diagram.

4.19 RANDOM FOREST: MODEL NOTES

Random Forest is an ensemble of decision trees trained with randomness in the sampled training examples and/or feature selection. The trees' predictions are combined to produce a final class. The ensemble often reduces the instability of an individual tree, although its effectiveness depends on data representation and parameters.

Sparse text matrices may be high-dimensional, and the performance of a forest can depend on the number of trees, maximum depth, and computational constraints. These settings should be recorded from the final code.

Inputs and output

Input: vectorised job-title features.

Target: job category.

Output: predicted category from the ensemble.

Evaluation

Report accuracy and the numerical confusion matrix.

Compare training and test performance only if both are actually computed.

Avoid attributing the score to one parameter without a controlled experiment.

4.20 K-NEAREST NEIGHBOURS: MODEL NOTES

K-Nearest Neighbours classifies a new observation by examining nearby training examples in feature space. For text represented by token counts, the meaning of “nearby” depends on the representation and distance metric. High-dimensional sparse text can make neighbourhoods less intuitive and computationally expensive.

The selected number of neighbours and any distance settings should be recorded. A model may perform differently from linear or probabilistic classifiers because it relies on local similarities rather than learning a single set of linear decision weights.

Inputs and output

Input: vectorised job title.

Target: category.

Output: the class selected from nearby training examples.

Evaluation

Report held-out test accuracy.

Inspect which classes are confused in the numerical matrix.

Compare with the other models under the same split.

4.21 FAIR COMPARISON OF THE FIVE MODELS

A fair model comparison keeps the dataset subset, target labels, text representation, and test split consistent. If different models are evaluated on different test records, accuracy values cannot be interpreted as a direct head-to-head comparison.

Accuracy is an appropriate simple baseline metric, but class imbalance can make it incomplete. Confusion matrices help reveal whether a model is strong on common classes but weak on less frequent classes. If additional metrics are introduced, they should be computed consistently and documented.

Comparison procedure

Use the same train-test split for each classifier.

Fit each model on the training feature matrix.

Predict labels for the same test feature matrix.

Calculate accuracy and confusion matrix.

Plot model names against their actual accuracy values.

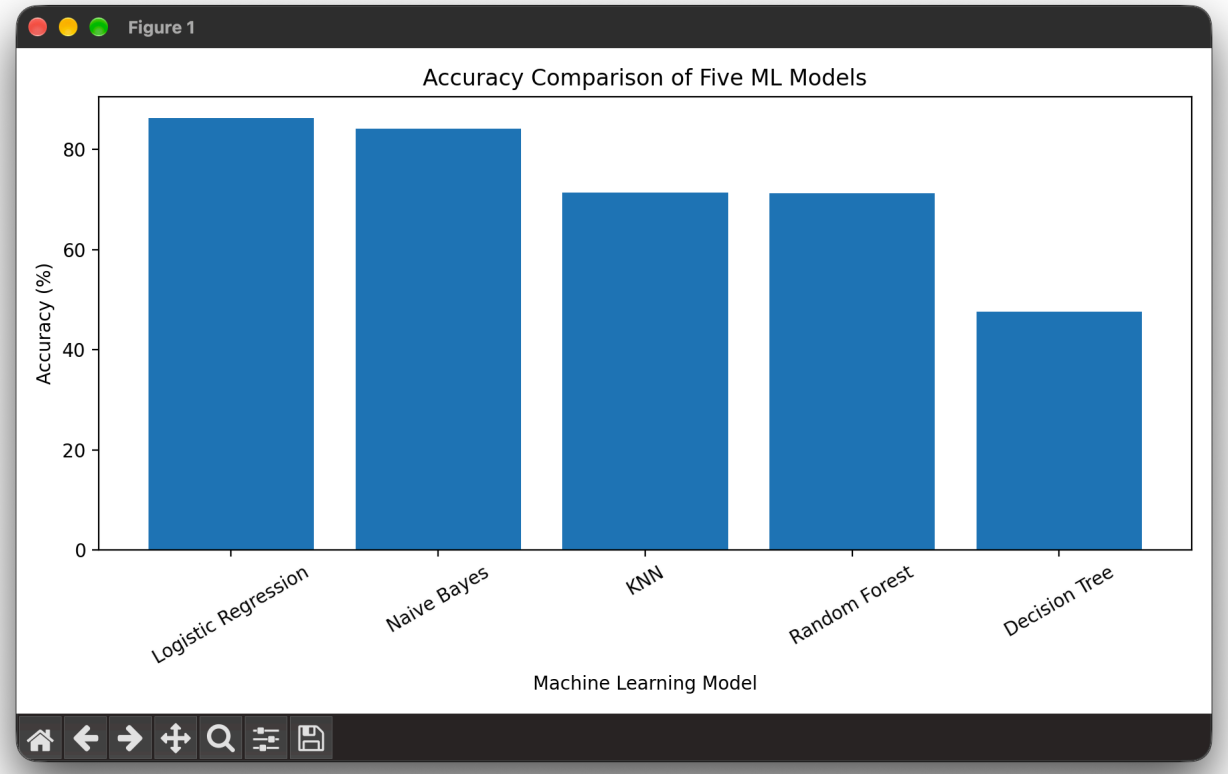
Result presentation

List the numerical scores in a table.

State which model is highest and lowest in this experiment.

Discuss the size of the difference without overgeneralising.

Retain the comparison chart as visual evidence.



4.22 LIMITATIONS OF THE ANALYSIS

The analysis is limited by the source dataset, the quality of its labels, the time period represented, and the choices made during cleaning. The dataset may not include all job portals, industries, regions, or employer types. The category label is treated as ground truth even though it may be noisy or inconsistent.

The title classifier uses only job title text. It does not use full descriptions, skills, location, experience requirements, salary, or employer context unless these are explicitly added to the feature set. As a result, two roles with similar titles but different responsibilities may be difficult to distinguish.

Technical limitations

CountVectorizer ignores word order beyond the tokenisation scheme.

Rare classes may have too few examples.

A random split may not reflect future job listings.

Accuracy alone does not measure all forms of classification quality.

Interpretation limitations

Counts describe this dataset, not necessarily the entire job market.

Remote flags may be incomplete.

Daily jumps may reflect data pipeline changes.

Company and skill counts depend on the source's aggregation logic.

4.23 ETHICAL AND RESPONSIBLE USE

Job-market data can influence career planning and recruitment decisions. The results should therefore be communicated carefully. A model trained on historical listings may reproduce biases in how categories were assigned or which employers were represented. It should not be used as the sole basis for employment decisions.

The project is intended as an educational analysis. It does not determine a candidate's suitability, guarantee job availability, or replace review of an actual job advertisement. Users should check original listings for eligibility, location, experience, compensation, and application requirements.

Responsible reporting

Describe the dataset and limitations transparently.

Do not claim that model predictions are guaranteed or unbiased.

Avoid presenting dataset counts as live vacancies unless the source confirms that status.

Respect the dataset licence and applicable platform terms.

4.24 REPRODUCIBILITY AND EXECUTION ENVIRONMENT

Reproducibility means that another person can rerun the same workflow and understand how the outputs were produced. The report should include the Python version, major library versions, file structure, execution steps, and any model parameters that materially affect results.

Because file paths and installed packages vary between computers, the code should use clear paths and produce readable error messages when an input file or required column is missing. Keep the code submitted with the report identical to the version used to generate the final screenshots.

Environment record

Operating system: [enter actual system].

Python version: [enter version].

pandas version: [enter version].

Matplotlib version: [enter version].

scikit-learn version: [enter version].

Suggested folder structure

project_folder/
  jobs.csv
  jobs_by_company.csv
  open_jobs_daily.csv
  skill_demand_daily.csv
  Job_Market_Analysis.py
  output/

5. CONCLUSION

This project presents a structured application of exploratory data analysis and supervised machine learning to a job-listings dataset. The work is organised around category distribution, fresher-friendly listings, company-level fresher roles, daily open-job trends, skill demand, remote-work flags, and prediction of job categories from job titles.

The EDA stage makes the source data easier to understand by examining its structure, checking data quality, applying documented filters, calculating group counts, and presenting results through bar and line charts. The analysis also illustrates the importance of validating field definitions before interpreting counts, especially for fresher-friendly flags, remote indicators, and daily aggregated measures.

The classification stage treats job-title category prediction as a multiclass text problem. CountVectorizer transforms titles into numerical token-count features, after which Logistic Regression, Multinomial Naive Bayes, Decision Tree, Random Forest, and KNN are compared using held-out accuracy and numerical confusion matrices. The actual numerical results must be copied from the final execution of the submitted code.

Overall, the project demonstrates an end-to-end workflow from raw CSV files to visual summaries and baseline predictive models. Its value lies in reproducible analysis and transparent reporting rather than claiming to describe the entire job market. Results should be interpreted with attention to dataset coverage, aggregation methods, class imbalance, and the fact that title-only classification has limited context.

Key outcomes to report after final execution

- Total number of records and columns in each source file.
- Number of records retained after cleaning and filtering.
- Leading job categories and fresher-friendly categories.
- Top companies by reported fresher-role counts.
- Observed daily open-job pattern and any discontinuities.
- Top skill labels by the source's defined demand measure.
- Remote and non-remote/not-flagged counts.
- Accuracy and confusion matrix for each classifier.
- Best-performing model on the defined test split and limitations of that comparison.

6. FUTURE SCOPE

6.1 Improve Dataset Coverage

Future work can combine multiple reputable job sources, provided their terms of use permit it. Sources should be deduplicated and documented. Region, industry, date, and source-platform filters could make the analysis more useful for a specific audience.

6.2 Add More Job Features

The classifier could use job descriptions, required skills, seniority terms, experience requirements, location, employment type, and employer information where these fields are available and appropriate. Richer features may improve classification but require more preprocessing and careful validation.

6.3 Explore Improved Text Representations

Possible extensions include TF-IDF features, word and character n-grams, and pretrained language representations. Each representation should be evaluated with the same train-test split and reported transparently.

6.4 Expand Model Evaluation

Future evaluation may include macro-averaged precision, recall, and F1-score, along with per-class performance. These measures can help evaluate minority classes that may be hidden by overall accuracy. Any additional metric should be selected to answer a clear question and computed from the same test predictions.

6.5 Time-Aware Evaluation

If dates are available for individual job listings, a time-based split could evaluate whether a model trained on earlier listings generalises to later listings. This is often more realistic than a random split when the intended use is future job categorisation.

6. FUTURE SCOPE (CONTINUED)

6.6 Interactive Dashboard

A future version could provide an interactive dashboard with filters for category, company, remote status, date range, and skills. A dashboard should show the data refresh date and make the source definitions visible so users can distinguish current listings from historical summaries.

6.7 Job Recommendation and Search

The title-classification component could be extended into a search assistant that groups similar roles or suggests related categories. Such a tool should explain that recommendations are based on available data and should link users back to original listings rather than imply that a role is guaranteed to be available.

6.8 Fresher-Focused Insights

If the source provides reliable experience-level information, future work could compare entry-level opportunities across categories, companies, locations, and skills. A carefully defined entry-level indicator is essential; job-title keywords alone may incorrectly classify internships, trainee positions, and experienced roles.

6.9 Automated Data Validation

A data-validation stage could check schema changes, unexpected data types, missing columns, duplicated dates, invalid counts, and new category labels before generating charts. This would reduce the risk of producing misleading output after the source dataset changes.

6.10 Deployment and Monitoring

A later iteration could be packaged as a small web application or dashboard. If the classifier is deployed, monitoring should track changes in category frequencies, vocabulary, and prediction quality. The model should be retrained only after review of the new data and an evaluation procedure.

6.11 Final Future-Work Priorities

- Verify dataset definitions and improve documentation.
- Add evaluation measures suited to class imbalance.
- Test time-based generalisation if dates are available.
- Compare count-based and TF-IDF text representations.
- Build an interactive dashboard with clear data provenance.
- Monitor data quality and category drift.

7. REFERENCES

Replace or complete the entries below with the exact pages and versions actually used. The dataset URL and date accessed must be verified from the original source.

[1] Kaggle dataset author, “Job Listings: Open Jobs in Every Field (Daily),” Kaggle. Exact dataset URL: https://www.kaggle.com/datasets/chandranshbinjola/job-listings/data

[2] pandas development team, “pandas documentation.” https://pandas.pydata.org/docs/

[3] NumPy developers, “NumPy documentation.” https://numpy.org/doc/

[4] Matplotlib development team, “Matplotlib documentation.” https://matplotlib.org/stable/

[5] scikit-learn developers, “scikit-learn documentation.” https://scikit-learn.org/stable/

[6] scikit-learn developers, “CountVectorizer documentation.” https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

[7] scikit-learn developers, “LogisticRegression documentation.” https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

[8] scikit-learn developers, “MultinomialNB documentation.” https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html

[9] scikit-learn developers, “DecisionTreeClassifier documentation.” https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

[10] scikit-learn developers, “RandomForestClassifier documentation.” https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[11] scikit-learn developers, “KNeighborsClassifier documentation.” https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

[12] scikit-learn developers, “Model evaluation: quantifying the quality of predictions.” https://scikit-learn.org/stable/modules/model_evaluation.html

Date accessed for online documentation: [enter actual date].

APPENDIX A. COMPLETE PYTHON CODE

```
# ============================================================
# GLOBAL JOB MARKET ANALYSIS
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer

from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import confusion_matrix, accuracy_score


# ============================================================
# 1. LOAD THE FOUR DATASETS
# ============================================================

jobs = pd.read_csv('/Users/sachinraj/Desktop/Python II/Project/archive/jobs.csv')
company = pd.read_csv('/Users/sachinraj/Desktop/Python II/Project/archive/jobs_by_company.csv')
daily = pd.read_csv('/Users/sachinraj/Desktop/Python II/Project/archive/open_jobs_daily.csv')
skills = pd.read_csv('/Users/sachinraj/Desktop/Python II/Project/archive/skill_demand_daily.csv')


print("\n--- JOBS DATASET ---")
print(jobs.head())
print(jobs.info())

print("\n--- COMPANY DATASET ---")
print(company.head())

print("\n--- DAILY JOBS DATASET ---")
print(daily.head())

print("\n--- SKILL DEMAND DATASET ---")
print(skills.head())


# ============================================================
# 2. DATA CLEANING
```

```
# ============================================================

# Remove duplicate rows from each dataset
jobs = jobs.drop_duplicates()
company = company.drop_duplicates()
daily = daily.drop_duplicates()
skills = skills.drop_duplicates()

# Remove rows without job title or category
jobs = jobs.dropna(subset=['title', 'category'])

# Clean text by converting it to lowercase and removing spaces
jobs['title'] = jobs['title'].astype(str).str.lower().str.strip()
jobs['category'] = jobs['category'].astype(str).str.lower().str.strip()

# Remove blank titles/categories
jobs = jobs[(jobs['title'] != '') & (jobs['category'] != '')]

# Remove the general 'other' category
jobs = jobs[jobs['category'] != 'other']

print("\nJobs dataset shape after cleaning:", jobs.shape)
print("\nJob categories:")
print(jobs['category'].value_counts())



# ============================================================
# OBJECTIVE 1: FIND THE MOST COMMON JOB CATEGORIES
# Dataset: jobs.csv
# ============================================================

print("\nOBJECTIVE 1: Most common job categories")

category_count = jobs['category'].value_counts().head(10)
print(category_count)

category_count.plot(kind='bar', figsize=(9, 5))
plt.title('Top 10 Job Categories')
plt.xlabel('Job Category')
plt.ylabel('Number of Job Listings')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()



# ============================================================
# OBJECTIVE 2: FIND CATEGORIES WITH FRESHER-FRIENDLY JOBS
# Dataset: jobs.csv
```

```
# ============================================================

print("\nOBJECTIVE 2: Fresher-friendly jobs by category")

if 'fresher_friendly' in jobs.columns:
    # Keep records marked as fresher-friendly
    fresher_jobs = jobs[
        jobs['fresher_friendly'].astype(str).str.lower().isin(
            ['true', '1', 'yes']
        )
    ]

    fresher_count = fresher_jobs['category'].value_counts().head(10)
    print(fresher_count)

    fresher_count.plot(kind='bar', figsize=(9, 5))
    plt.title('Top Categories for Fresher-Friendly Jobs')
    plt.xlabel('Job Category')
    plt.ylabel('Number of Jobs')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
else:
    print("The fresher_friendly column was not found in jobs.csv.")


# ============================================================
# OBJECTIVE 3: FIND COMPANIES WITH THE MOST FRESHER ROLES
# Dataset: jobs_by_company.csv
# ============================================================

print("\nOBJECTIVE 3: Companies offering fresher roles")

if 'company' in company.columns and 'fresher_roles' in company.columns:
    company['fresher_roles'] = pd.to_numeric(
        company['fresher_roles'], errors='coerce'
    )
    company = company.dropna(subset=['fresher_roles'])
    top_companies = company.sort_values(
        by='fresher_roles', ascending=False
    ).head(10)

    print(top_companies[['company', 'fresher_roles']])

    plt.figure(figsize=(10, 5))
    plt.bar(top_companies['company'].astype(str),
            top_companies['fresher_roles'])
    plt.title('Top 10 Companies Offering Fresher Roles')
```

```
    plt.xlabel('Company')
    plt.ylabel('Number of Fresher Roles')
    plt.xticks(rotation=60)
    plt.tight_layout()
    plt.show()
else:
    print("Expected columns 'company' and 'fresher_roles' were not found.")
    print("Available columns:", list(company.columns))


# ============================================================
# OBJECTIVE 4: VISUALISE DAILY OPEN-JOB TRENDS
# Dataset: open_jobs_daily.csv
# ============================================================

print("\nOBJECTIVE 4: Daily open-job trends")

if 'date' in daily.columns and 'open_jobs' in daily.columns:
    daily['date'] = pd.to_datetime(daily['date'], errors='coerce')
    daily['open_jobs'] = pd.to_numeric(daily['open_jobs'], errors='coerce')
    daily = daily.dropna(subset=['date', 'open_jobs'])
    daily = daily.sort_values('date')

    plt.figure(figsize=(10, 5))
    plt.plot(daily['date'], daily['open_jobs'], marker='o')
    plt.title('Daily Open Job Trend')
    plt.xlabel('Date')
    plt.ylabel('Number of Open Jobs')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()

    if 'fresher_friendly_jobs' in daily.columns:
        daily['fresher_friendly_jobs'] = pd.to_numeric(
            daily['fresher_friendly_jobs'], errors='coerce'
        )
        plt.figure(figsize=(10, 5))
        plt.plot(daily['date'], daily['open_jobs'], label='All Open Jobs')
        plt.plot(daily['date'], daily['fresher_friendly_jobs'],
                 label='Fresher-Friendly Jobs')
        plt.title('Daily Open Jobs: All vs Fresher-Friendly')
        plt.xlabel('Date')
        plt.ylabel('Number of Jobs')
        plt.legend()
        plt.xticks(rotation=45)
        plt.tight_layout()
        plt.show()
else:
```

```
    print("Expected columns 'date' and 'open_jobs' were not found.")
    print("Available columns:", list(daily.columns))


# ============================================================
# OBJECTIVE 5: FIND THE MOST DEMANDED SKILLS
# Dataset: skill_demand_daily.csv
# ============================================================

print("\nOBJECTIVE 5: Most demanded skills")

if 'skill' in skills.columns and 'jobs' in skills.columns:
    skills['jobs'] = pd.to_numeric(skills['jobs'], errors='coerce')
    skills = skills.dropna(subset=['skill', 'jobs'])
    top_skills = skills.groupby('skill')['jobs'].sum().sort_values(
        ascending=False
    ).head(10)

    print(top_skills)

    top_skills.sort_values().plot(kind='barh', figsize=(9, 5))
    plt.title('Top 10 Most Demanded Skills')
    plt.xlabel('Number of Jobs')
    plt.ylabel('Skill')
    plt.tight_layout()
    plt.show()
else:
    print("Expected columns 'skill' and 'jobs' were not found.")
    print("Available columns:", list(skills.columns))


# ============================================================
# OBJECTIVE 6: COMPARE REMOTE AND NON-REMOTE JOBS
# Dataset: jobs.csv
# ============================================================

print("\nOBJECTIVE 6: Remote and non-remote jobs")

if 'remote' in jobs.columns:
    remote_count = jobs['remote'].astype(str).value_counts()
    print(remote_count)

    remote_count.plot(kind='bar', figsize=(7, 4))
    plt.title('Remote vs Other Job Listings')
    plt.xlabel('Remote Value')
    plt.ylabel('Number of Jobs')
    plt.xticks(rotation=0)
    plt.tight_layout()
```

```
    plt.show()
else:
    print("The remote column was not found in jobs.csv.")


# ============================================================
# OBJECTIVE 7: PREDICT JOB CATEGORY USING MACHINE LEARNING
# Input (X): Job title
# Output (y): Job category
# ============================================================

print("\nOBJECTIVE 7: Machine Learning - Job Category Prediction")

# Use the eight most common categories to keep training manageable
top_categories = jobs['category'].value_counts().head(8).index
ml_data = jobs[jobs['category'].isin(top_categories)][
    ['title', 'category']
].copy()

# Use up to 2,500 rows to keep the program manageable in IDLE
ml_data = ml_data.sample(
    n=min(2500, len(ml_data)),
    random_state=42
)

print("\nMachine Learning data shape:", ml_data.shape)
print("\nCategories used:")
print(ml_data['category'].value_counts())

# Independent variable: job title
X = ml_data['title']

# Dependent variable: job category
y = ml_data['category']

# Split the dataset into training and testing sets
X_train_text, X_test_text, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# Convert job-title text into numeric word-count features
vectorizer = CountVectorizer(max_features=300, stop_words='english')
X_train = vectorizer.fit_transform(X_train_text)
X_test = vectorizer.transform(X_test_text)
```

```
# ============================================================
# MODEL 1: LOGISTIC REGRESSION
# ============================================================

print("\nMODEL 1: LOGISTIC REGRESSION")

lr_model = LogisticRegression(max_iter=1000)

# Train the model
lr_model.fit(X_train, y_train)

# Predict job categories for test data
lr_pred = lr_model.predict(X_test)

# Calculate and display accuracy
lr_accuracy = accuracy_score(y_test, lr_pred)
print("Accuracy:", round(lr_accuracy, 4))
print("Accuracy (%):", round(lr_accuracy * 100, 2))

# Display confusion matrix as numbers (not a graph)
print("Confusion Matrix:")
print(confusion_matrix(y_test, lr_pred, labels=lr_model.classes_))



# ============================================================
# MODEL 2: NAIVE BAYES
# ============================================================

print("\nMODEL 2: NAIVE BAYES")

nb_model = MultinomialNB()

# Train the model
nb_model.fit(X_train, y_train)

# Predict job categories for test data
nb_pred = nb_model.predict(X_test)

# Calculate and display accuracy
nb_accuracy = accuracy_score(y_test, nb_pred)
print("Accuracy:", round(nb_accuracy, 4))
print("Accuracy (%):", round(nb_accuracy * 100, 2))

# Display confusion matrix as numbers (not a graph)
print("Confusion Matrix:")
print(confusion_matrix(y_test, nb_pred, labels=nb_model.classes_))
```

```
# ============================================================
# MODEL 3: DECISION TREE CLASSIFIER
# Follows the structure used in the classroom example.
# ============================================================

print("\nMODEL 3: DECISION TREE CLASSIFIER")

# Create Decision Tree Classifier
dt_model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42,
    max_depth=6
)

# Train the model
dt_model.fit(X_train, y_train)

# Predict test data
dt_pred = dt_model.predict(X_test)

# Confusion Matrix
dt_cm = confusion_matrix(y_test, dt_pred, labels=dt_model.classes_)

print("Confusion Matrix:")
print(dt_cm)

# Accuracy
dt_accuracy = accuracy_score(y_test, dt_pred)

print("Accuracy:", dt_accuracy)
print("Accuracy (%):", dt_accuracy * 100)

# Visualise the Decision Tree
# max_depth=3 limits the displayed tree to make it easier to read.
# It does not change the trained model.
plt.figure(figsize=(22, 10))

plot_tree(
    dt_model,
    max_depth=3,
    feature_names=vectorizer.get_feature_names_out(),
    class_names=[str(name) for name in dt_model.classes_],
    filled=True,
    rounded=True,
    fontsize=7
)
```

```
plt.title("Decision Tree Classifier")
plt.tight_layout()
plt.show()


# ============================================================
# MODEL 4: RANDOM FOREST CLASSIFIER
# ============================================================

print("\nMODEL 4: RANDOM FOREST CLASSIFIER")

rf_model = RandomForestClassifier(
    n_estimators=50,
    max_depth=12,
    random_state=42,
    n_jobs=-1
)

# Train the model
rf_model.fit(X_train, y_train)

# Predict test data
rf_pred = rf_model.predict(X_test)

# Calculate and display accuracy
rf_accuracy = accuracy_score(y_test, rf_pred)
print("Accuracy:", round(rf_accuracy, 4))
print("Accuracy (%):", round(rf_accuracy * 100, 2))

# Display confusion matrix as numbers (not a graph)
print("Confusion Matrix:")
print(confusion_matrix(y_test, rf_pred, labels=rf_model.classes_))


# ============================================================
# MODEL 5: K-NEAREST NEIGHBOURS (KNN)
# ============================================================

print("\nMODEL 5: K-NEAREST NEIGHBOURS")

knn_model = KNeighborsClassifier(n_neighbors=5)

# Train the model
knn_model.fit(X_train, y_train)

# Predict test data
knn_pred = knn_model.predict(X_test)
```

```
# Calculate and display accuracy
knn_accuracy = accuracy_score(y_test, knn_pred)
print("Accuracy:", round(knn_accuracy, 4))
print("Accuracy (%):", round(knn_accuracy * 100, 2))

# Display confusion matrix as numbers (not a graph)
print("Confusion Matrix:")
print(confusion_matrix(y_test, knn_pred, labels=knn_model.classes_))



# ============================================================
# FINAL MODEL COMPARISON
# ============================================================

comparison = pd.DataFrame({
    'Model': [
        'Logistic Regression',
        'Naive Bayes',
        'Decision Tree',
        'Random Forest',
        'KNN'
    ],
    'Accuracy': [
        lr_accuracy,
        nb_accuracy,
        dt_accuracy,
        rf_accuracy,
        knn_accuracy
    ]
})

comparison = comparison.sort_values(
    by='Accuracy',
    ascending=False
)

print("\nFINAL MODEL COMPARISON")
print(comparison.to_string(index=False))

# Save results to a CSV file
comparison.to_csv('model_comparison.csv', index=False)

# Compare accuracy of all five models in a bar chart
plt.figure(figsize=(9, 5))
plt.bar(comparison['Model'], comparison['Accuracy'] * 100)
plt.title('Accuracy Comparison of Five ML Models')
plt.xlabel('Machine Learning Model')
plt.ylabel('Accuracy (%)')
```

```
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

# Display the best model according to accuracy
best_model = comparison.iloc[0]['Model']
best_accuracy = comparison.iloc[0]['Accuracy']

print("\nBest model based on accuracy:", best_model)
print("Best accuracy (%):", round(best_accuracy * 100, 2))

print("\nPROJECT COMPLETED")
```

6. GitHub and Linkedin

Github: https://github.com/SachinRaj0/feature-engineering---Data-Science-and-Machine-Learning-project-using-Python

Linkedin: https://lnkd.in/p/g_9gAsd6